

wyndctl

Command Reference & Technical Guide

wyndctl is a Go CLI for scanning and deploying Wynd Sentry IoT air-quality devices over USB serial. It exposes three commands — **scan**, **deploy**, and **delete-device** — and connects to a Postgres database to register and manage device assignments in the Wynd platform.

Global Flags

These flags are available on every command and are set once for the entire session.

Flag	Default	Description
--config	./wyndctl.yaml	Path to the config file. Searched in current directory, home directory, and /etc/wynd.
--env	dev	Deployment environment: dev, staging, or prod. Controls which DB secret and IoT resources are used.
--log-level	info	Log verbosity: debug, info, warn, error.
--log-format	text	Log output format: text (human-readable) or json (structured, for log aggregators).

Tip: set env permanently by creating ~/.wyndctl.yaml with `env: prod`, or export `WYND_ENV=prod` in your shell profile.

scan — Discover Devices

Finds all Sentry devices connected via USB, prints their firmware versions, and lights their LED indicators. With **--label**, interactively maps each device to a bay number and writes location-map.csv.

```
wyndctl scan [--label] [--color R,G,B]
```

Flags

Flag	Default	Description
--label	false	Interactive labeling mode. Lights each device's LED and prompts the operator to type a bay number. Writes bay, USB location, and device ID to location-map.csv.
--color R,G,B	255,165,0	RGB color for the device LED indicator. Orange by default. Example: 255,0,0 for red.
-o / --output FMT	text	Output format: text (human-readable) or json (structured). Defaults to json automatically when --log-format json is set.

What Happens Behind the Scenes

Step 1 — USB Discovery

The CLI calls `transport.FindSerialPorts()` which scans all system serial ports and filters by USB Vendor ID 0x2FE3 and Product ID 0x0100 — the hardware IDs burned into every Sentry device. Only matching ports are returned.

Step 2 — Channel Activation

Each matched port is opened as a `SerialChannel` at 9600 baud. The COBS framing layer is initialised on top: all data is encoded so the 0x00 byte acts as a clean packet delimiter.

Step 3 — LED Reset

Before printing any info, the CLI sends a `CancelIndication` RPC to every device, turning off any previously set LED state. A 1.5-second pause follows each reset to let the hardware settle.

Step 4 — GetDeviceInfo RPC

For each channel, the CLI encodes a request using the hand-rolled protobuf-compatible encoder and calls `RPC.UnaryCall(packageID=1, serviceID=1, methodID=1)`. The device responds with its AWS Thing name, firmware version, WiFi firmware version, and PM firmware version.

Step 5 — LED Indication

After printing device info, the CLI calls `SetIndicate` with the `--color` value. In `--label` mode only one device LED is lit at a time so the operator can visually identify which physical device to assign a bay number to.

Step 6 — location-map.csv (--label only)

Each operator-entered bay number is written to `location-map.csv` as a row of bay, USB port path, device ID. This file is later consumed by the `deploy` command to match physical USB positions to deployment settings.

Output Files

Flag	Default	Description
location-map.csv	--label only	Columns: bay, location (USB port path), device_id. Used by deploy to match ports to settings.

deploy — Mass Provision Devices

Provisions multiple Sentry devices in parallel: configures WiFi credentials over USB, registers each device in AWS IoT, assigns it to a property and room in the Postgres database, and logs every result to deployment-result.csv.

```
wyndctl deploy [flags]
```

Flags

Flag	Default	Description
--all	false	Deploy all found devices. Without this flag, devices already marked succeeded in deployment-result.csv are skipped — allowing clean retries of failed devices only.
--color R,G,B	255,0,255	RGB color for the LED while a device is actively being deployed. Defaults to magenta.
--iterative	false	One-by-one mode. Lights a device LED and waits for the operator to press Enter before deploying it. Useful for manual verification.
--workers N	runtime.NumCPU()	Number of parallel goroutines. Defaults to the number of CPU cores on the machine running the CLI.
--timeout N	75	Per-device provisioning timeout in seconds. If the device does not reach ProvisionMQTTPublish status within this window, the attempt is counted as failed.

Required Input Files

Flag	Default	Description
deployment-data.csv	required	Columns: bay, wifi_ssid, wifi_psk, account, lodging_id, room, room_type. wifi_ssid, account, lodging_id, and room_type inherit from the previous row if left blank.
location-map.csv	optional	Created by scan --label. Maps USB port paths to bay numbers. If absent, devices are assigned bay numbers by discovery order.

What Happens Behind the Scenes

Step 1 — Load CSV Settings

deployment-data.csv is parsed row by row. Blank cells for wifi_ssid, account, lodging_id, and room_type inherit the value from the previous row, so a block of rooms in the same property only needs those fields set once. wifi_psk never inherits — it must be explicit.

Step 2 — Load Location Map

If location-map.csv exists, it is loaded into a map of USB port path to bay number. If the file is absent, devices are numbered in USB discovery order with a warning logged.

Step 3 — USB Scan

Identical to the scan command: FindSerialPorts filters by VID/PID, each port is opened as a SerialChannel at 9600 baud, and COBS framing is initialised on every channel.

Step 4 — Operator Confirmation

The CLI prompts Deploy N device(s) on ? [y/N] to stderr. The operator must type y or yes. Any other input cancels the run. This prompt cannot be bypassed.

Step 5 — Database Connection

The CLI connects to Postgres (credentials resolved from AWS Secrets Manager for dev/prod). This is optional — if the database is unavailable, room assignment is skipped with a warning and the rest of the deployment continues.

Step 6 — Parallel Deployment (default)

One goroutine is spawned per device, bounded by a semaphore of size --workers. Each goroutine runs the full provisioning sequence independently. With --iterative, a single goroutine runs each device in sequence after the operator presses Enter.

Step 7 — Unprovision

The CLI sends an Unprovision RPC to clear any existing WiFi and MQTT credentials from the device's flash storage. This ensures a clean state before writing new credentials. A 1-second sleep follows to let the device settle.

Step 8 — Disable BLE Advertising

SetAdvertising(false) is called to turn off Bluetooth advertising. This reduces RF interference during the WiFi provisioning handshake and is standard procedure before every deployment.

Step 9 — SetProvision — WiFi + MQTT

The CLI sends the WiFi SSID, PSK, and MQTT broker URL to the device via the SetProvision RPC. It then polls GetStatus every few seconds until the device reaches ProvisionMQTTPublish state (state 7), confirming the device connected to WiFi, resolved DNS, connected to the MQTT broker, and published its first message. If the --timeout window expires without reaching this state, the attempt is marked failed. Up to 5 retries are attempted with a device reboot between each.

Step 10 — Database — Room Assignment

The CLI calls GetUserByEmail to verify the account exists, GetLodging to confirm the lodging belongs to that user, then FindOrCreateZone to get or create the room record. Finally AssignDeviceToZone upserts the device row with the zone ID. If the user or lodging is not found, the deployment for that device is marked failed.

Step 11 — LED Feedback + Result Log

On success the LED is set to the --color value. On failure it is set to red (255,0,0). A row is appended to deployment-result.csv with bay, device_id, mac_addr, succeeded, room_id, room_name, and a failure reason if applicable.

Output Files

Flag	Default	Description
deployment-result.csv	always written	Columns: bay, device_id, mac_addr, succeeded, room_id, room_name, reason. Re-running without --all uses this file to skip already-succeeded devices.

delete-device — Remove Device Assignments

Removes device-to-room assignments from the database. Device rows and AWS IoT certificates are preserved so devices can be redeployed at any time. Pass **--delete-room** to also remove the associated zone records.

```
wyndctl delete-device --account EMAIL [--lodging-id N] [--device-id ID] [--delete-room]
```

Flags

Flag	Default	Description
--account EMAIL	required	Email address of the user whose devices will be cleared.
--lodging-id N	0 (all)	Scope deletion to a specific lodging ID. If omitted, all lodgings owned by the account are affected.

Flag	Default	Description
--device-id ID	(all)	Scope deletion to a single device by AWS Thing name. If omitted, all devices under the account (and lodging, if specified) are affected.
--delete-room	false	Also delete the associated zone/room records from the database. Without this flag only the device-to-zone link is cleared.

What Happens Behind the Scenes

Step 1 — Resolve User

The CLI looks up the user by --account email. If no matching user is found the command exits with an error immediately.

Step 2 — Query Devices

Devices are queried by joining device → zone → lodging, filtered by owner_id. Optionally narrowed by --lodging-id or --device-id. If no devices are found the command prints a message and exits cleanly.

Step 3 — Clear Zone Assignment

For each matching device, zone_id is set to NULL in the database. The device row itself is kept intact — the device can be redeployed to a new room at any time without any hardware or IoT reconfiguration.

Step 4 — Delete Room (--delete-room only)

If --delete-room is passed, the zone record associated with each device is also deleted from the database. Use with care — if multiple devices share a zone, the zone is deleted when the first device in it is processed.

Step 5 — Result

The CLI prints the count of cleared assignments and exits. No AWS IoT changes are made — certificates and policies are never modified.

Retry and Resume

If some devices fail, re-run **wyndctl deploy** without **--all**. The CLI reads the existing deployment-result.csv and skips any bay already marked succeeded=true, so only the failed devices are retried. Use **--all** to force a full re-deployment of every connected device regardless of prior results.

```
# First run – some devices fail
wyndctl deploy

# Retry only failures
wyndctl deploy
```

```
# Force re-deploy everything
wyndctl deploy --all
```

Provisioning State Machine

After SetProvision is called, the device moves through these states. The CLI polls until state 7 (ProvisionMQTTPublish) or the timeout expires.

State	Name	Meaning
0	ProvisionOff	Device is off / not started
1	ProvisionUnprovisioned	Credentials cleared, ready to receive new config
2	ProvisionWiFiWait	Credentials received, attempting WiFi association
3	ProvisionWiFiDisconnect	WiFi association failed, retrying
4	ProvisionWiFiConnect	WiFi associated successfully
5	ProvisionWiFiPing	DNS/connectivity check in progress
6	ProvisionMQTTConnect	Connecting to MQTT broker (AWS IoT endpoint)
7	ProvisionMQTTPublish	Connected and published — provisioning complete
8	ProvisionMQTTWait	Waiting for MQTT acknowledgement

Communication Stack

Every device interaction travels through four layers before reaching the physical USB port.

Layer	File	What it does
COBS Framing	transport/cobs.go	Encodes packets so 0x00 is always the frame delimiter. Guarantees clean boundaries on the serial stream.
Packet	transport/packet.go	Wraps encoded data in typed packets (ServerRequest, ClientResponse, etc.) with a CRC16 integrity checksum.

Layer	File	What it does
RPC	transport/rpc.go	Multiplexes unary calls identified by (packageID, serviceID, methodID, invocationID) over a single serial channel.
Encoding	device/encoding.go + proto.go	Hand-rolled protobuf-compatible wire encoding. Field IDs are hardcoded constants — no .proto files or code generation.